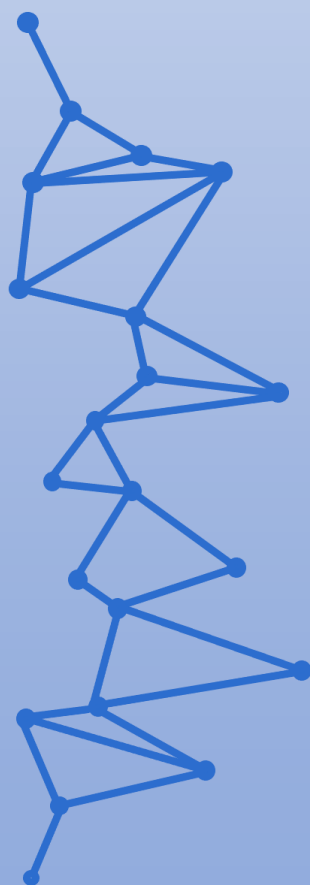




Curso de Especialización de Ciberseguridad en Entornos de Tecnología de la Información (CETI)



Análisis Forense Informático

UD01. Análisis de memoria RAM.
Tarea Online.

JUAN ANTONIO GARCIA MUELAS

INDICE

	Pag
1. Caso práctico	2
2. Analiza la memoria RAM	2
3. Contestando a las preguntas	4
4. Anexo	5
5. Webgrafía	17

1.- Descripción de la tarea.

Caso práctico

María está trabajando en un caso y ha recibido la imagen de memoria RAM de un servidor que ha tenido un comportamiento anómalo.

Un compañero sobre el terreno ha capturado la memoria RAM de la máquina antes de que la apagasen y se la ha enviado al laboratorio para ser analizada por María.

El coordinador de la investigación le ha hecho varias preguntas a María que deberá responder sobre la evidencia.

¿Qué te pedimos que hagas?

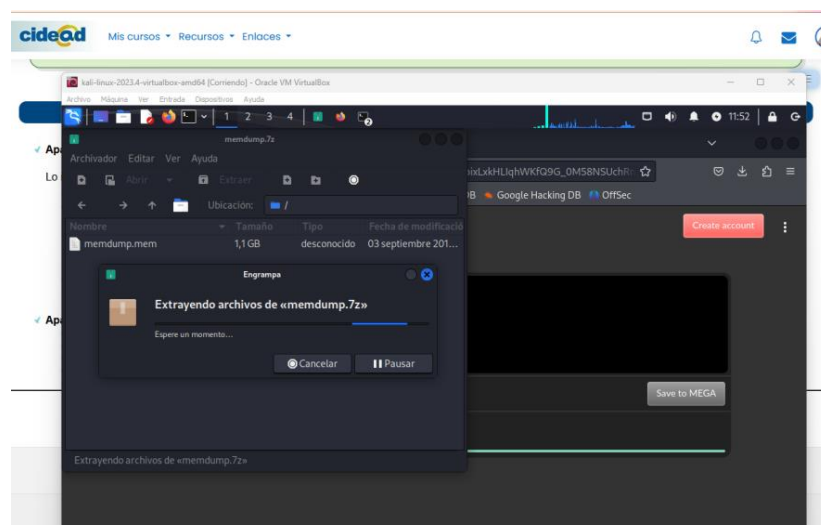
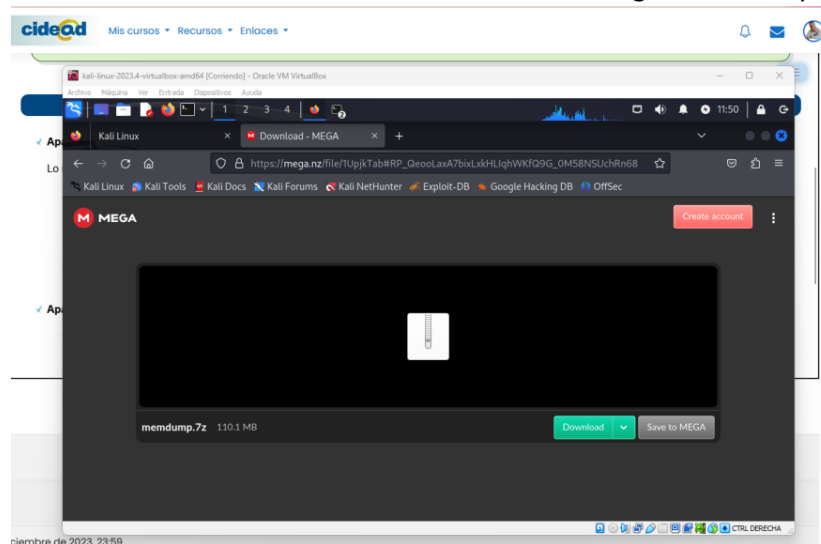
✓ Apartado 1: Analiza la memoria RAM

Lo ideal es usar la herramienta Volatility (versión 2), la tienes disponible en [Volatility](#)

- La memoria RAM está disponible en este enlace (https://mega.co.nz/#!1UpjkTab!RP_QeooLaxA7bixLxkHLIqhWKfQ9G_0M58NSUchRn68)

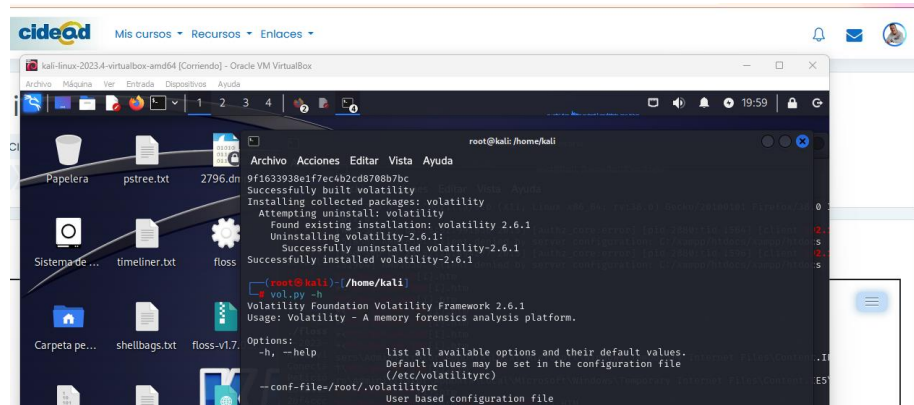
- Descomprime la memoria RAM

Accedo directamente desde una VM de Kali a descargar el archivo y a descomprimirlo.

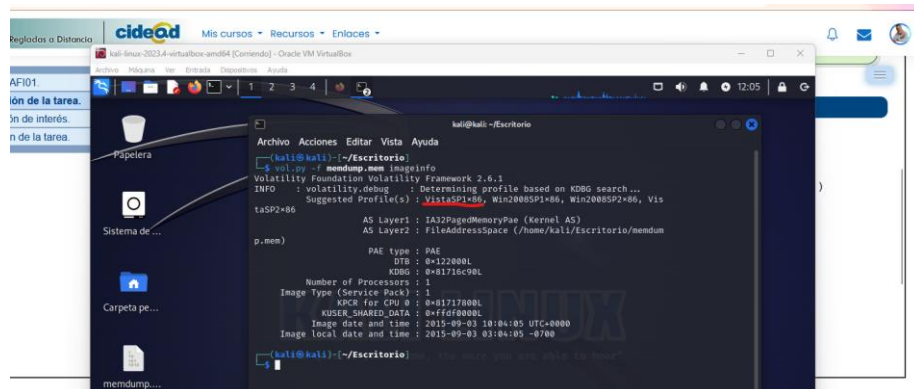


- Debes de ejecutar Volatility desde consola ya sea en Windows o Linux
- Tienes varias guías de vídeo que te pueden ayudar en el proceso:
 - <https://www.youtube.com/watch?v=RFYbevW6hxl>
 - <https://www.youtube.com/watch?v=iU9mqB4h3Tg>

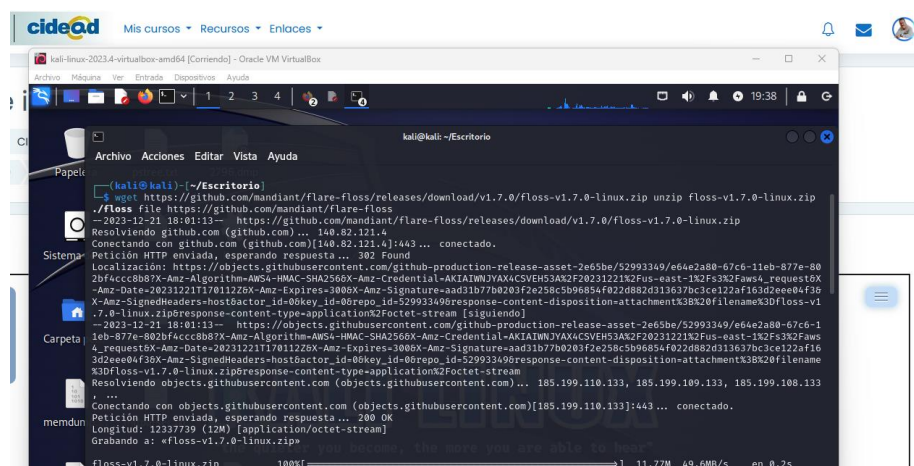
Tras la instalación comprobamos que podemos utilizar Volatility.

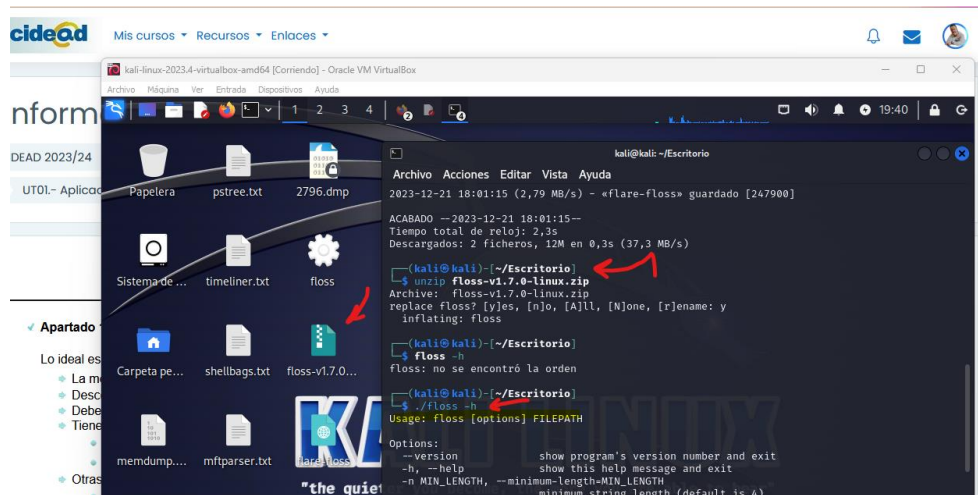


Comenzamos a ejecutar el análisis del volcado de memoria.



- Otras herramientas que te pueden ser útiles
 - Floss: <https://github.com/mandiant/flare-floss>





✓ Apartado 2: Contestando a las preguntas

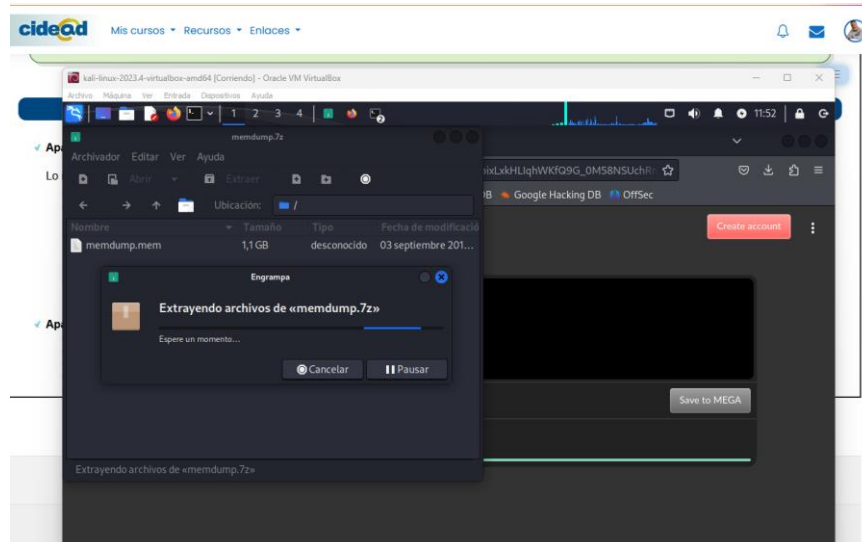
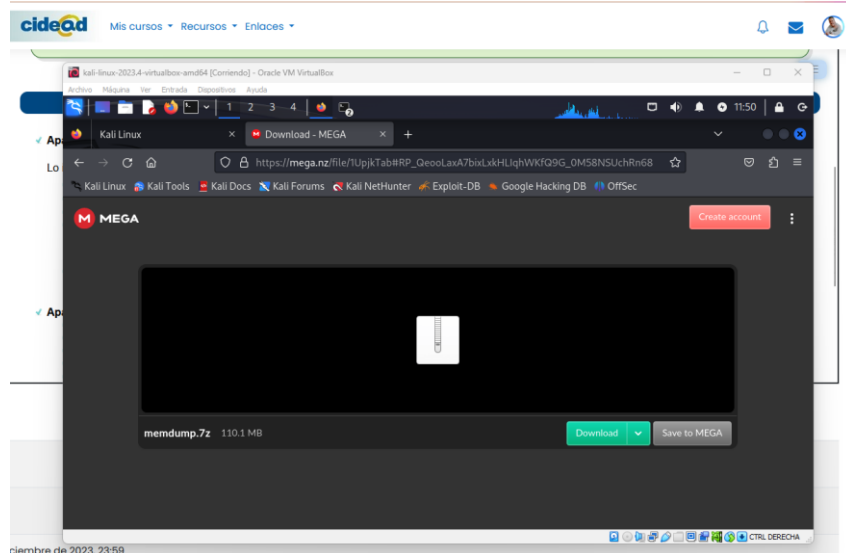
- ¿Qué pasaría si se hubiera apagado este servidor?
Al tratarse de algo tan volátil como la memoria RAM, habríamos perdido todas las pruebas, tanto a nivel de información como de línea temporal, de este fichero.
- ¿Qué tipo de comandos ha ejecutado el cibercriminal? ¿Qué sugiere?
Tiene acceso al sistema, pues ha podido ejecutar comandos por CMD (ha creado usuarios, creado reglas para permitir las conexiones, lo ha incluido en el protocolo de administración remota, ...) desde un escritorio remoto.
- ¿Cómo se han ejecutado los comandos?
Por acceso remoto desde una consola con derechos de administrador.
- ¿Qué actividad maliciosa has visto?
Ha conseguido mediante una webshell acceso al CMD y agregado un usuario con privilegios. Ha ejecutado procesos y ha subido ficheros infectados por malware. Con el c99.php ha seguido enviando webshells e invalidando los login.
- ¿Puedes identificar desde que IP vino el ataque?
192.168.56.102 como servidor de escucha desde DWVA, apuntando a la dirección del router 192.168.56.1.
- ¿Qué tipo de ataque pudo ser? ¿Qué tipo de malware se ha encontrado?
SQL Injection , XSS Reflected y subidas de archivos para desplegar la webshell c99.php.
Se han encontrado rastros de un troyano WIn32, además del malware TEHTRIS detectado por Virustotal.

ANEXO

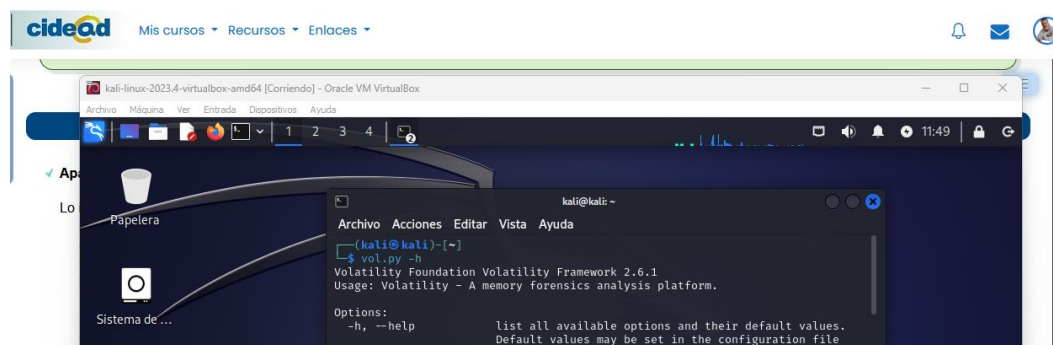
Apartado 1: Analiza la memoria RAM

Para la realización de esta tarea he utilizado una máquina virtual Kali Linux descargada desde su propio sitio web.

Lo primero que hacemos es descomprimir el archivo en nuestro escritorio.

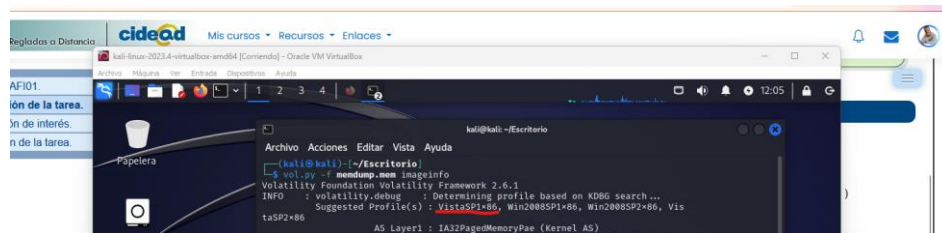


Con el archivo disponible, abrimos una consola y comprobamos el funcionamiento de la herramienta propuesta para realización de esta tarea: **Volatility**.



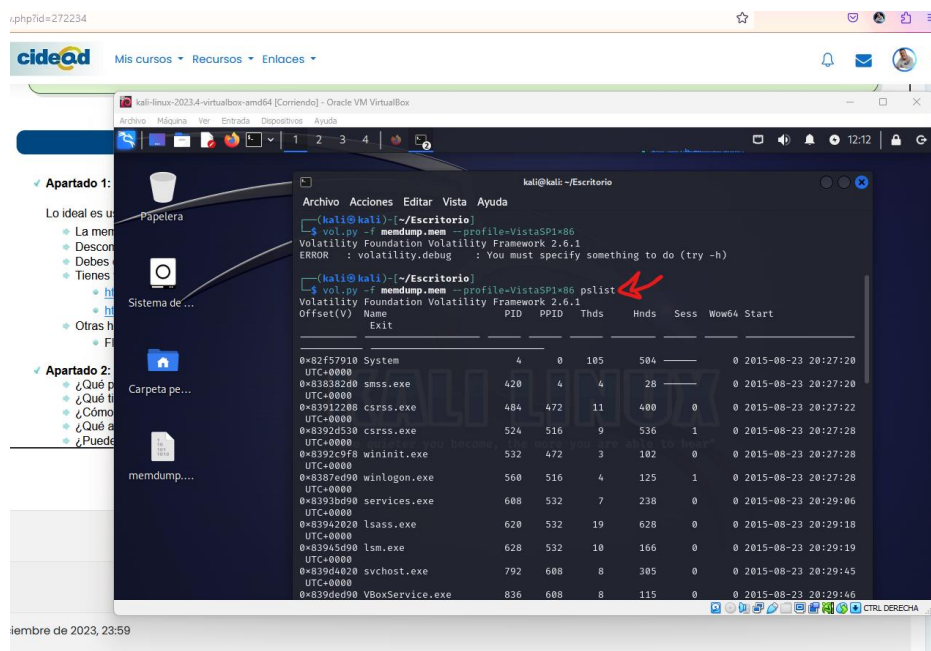
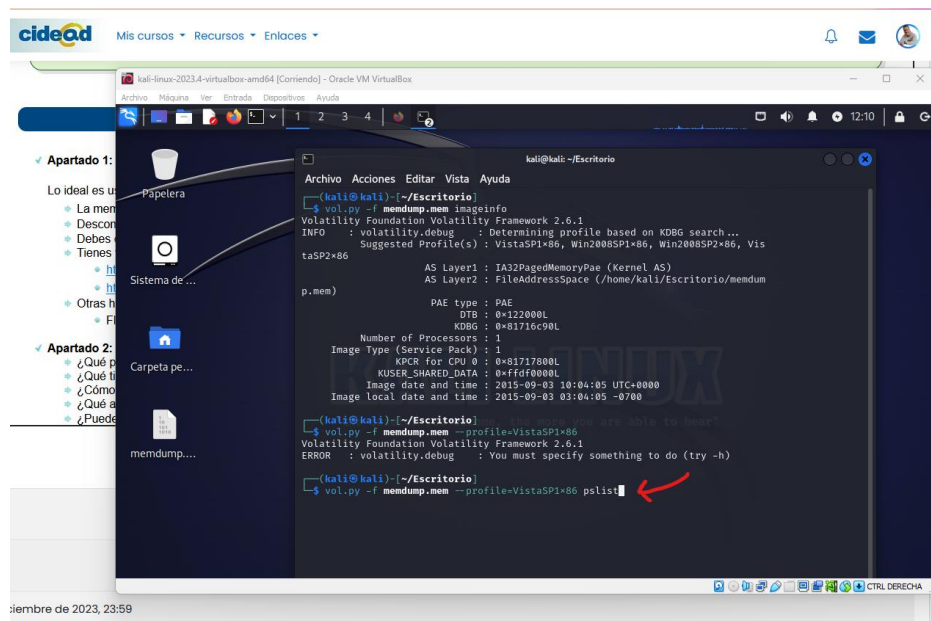
El primer paso será comprobar el sistema operativo mediante el comando:

vol.py -f memdump.mem imageinfo



Comprobado que estamos ante un Windows Vista (VistaSP1x86), observamos los procesos en ejecución:

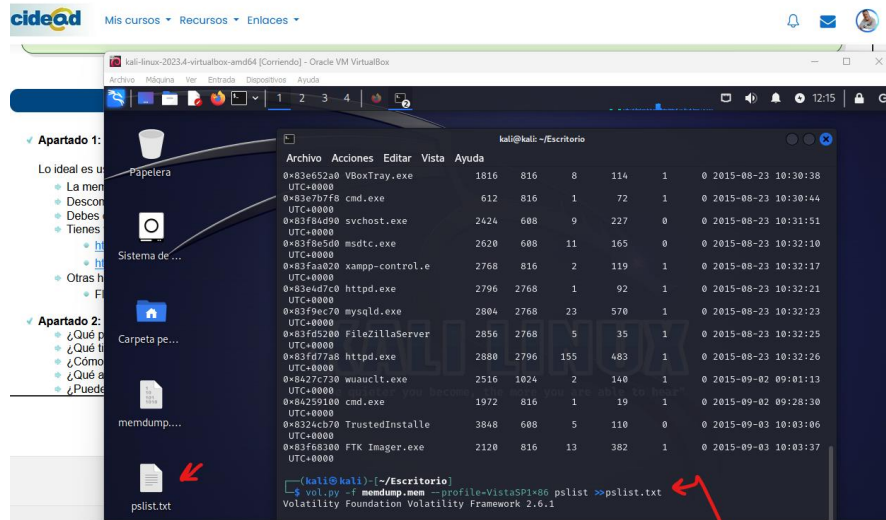
vol.py -f memdump.mem --profile=VistaSP1x86 pslist



Salтан a la vista el servidor **Xampp** y las herramientas que le acompañan (**mysqld**, **Filezilla**, **httpd**), o el propio **cmd**.

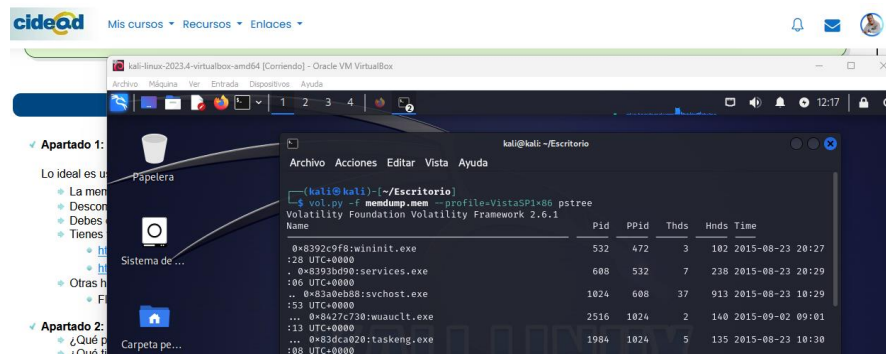
Para facilitar el análisis y la exportación de los datos extraídos, los exportamos a un archivo de texto.

```
vol.py -f memdump.mem -profile=VistaSP1x86 pslist >>pslist.txt
```

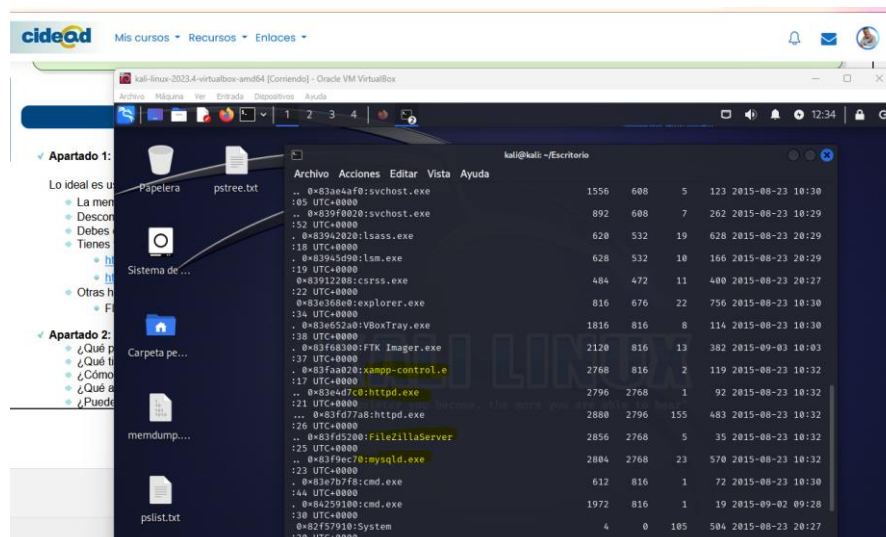


Vistos los procesos ejecutados y el momento de inicio, podemos buscar mayor detalle:

```
vol.py -f memdump.mem -profile=VistaSP1x86 pstree
```



Ahora podemos ver, en modo árbol, los procesos padre y los que cuelgan de ellos.

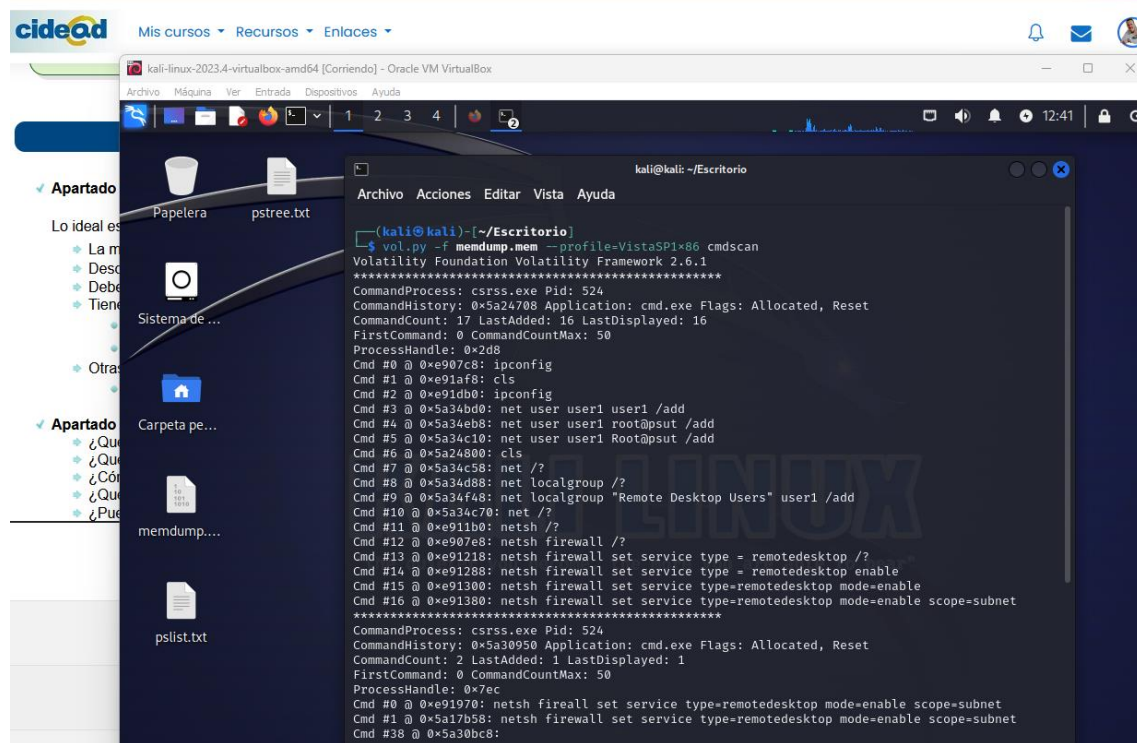


Igual que con el paso anterior, extraemos a un archivo de texto.

```
vol.py -f memdump.mem --profile=VistaSP1x86 pstree >>pstree.txt
```

Buscamos los comandos utilizados.

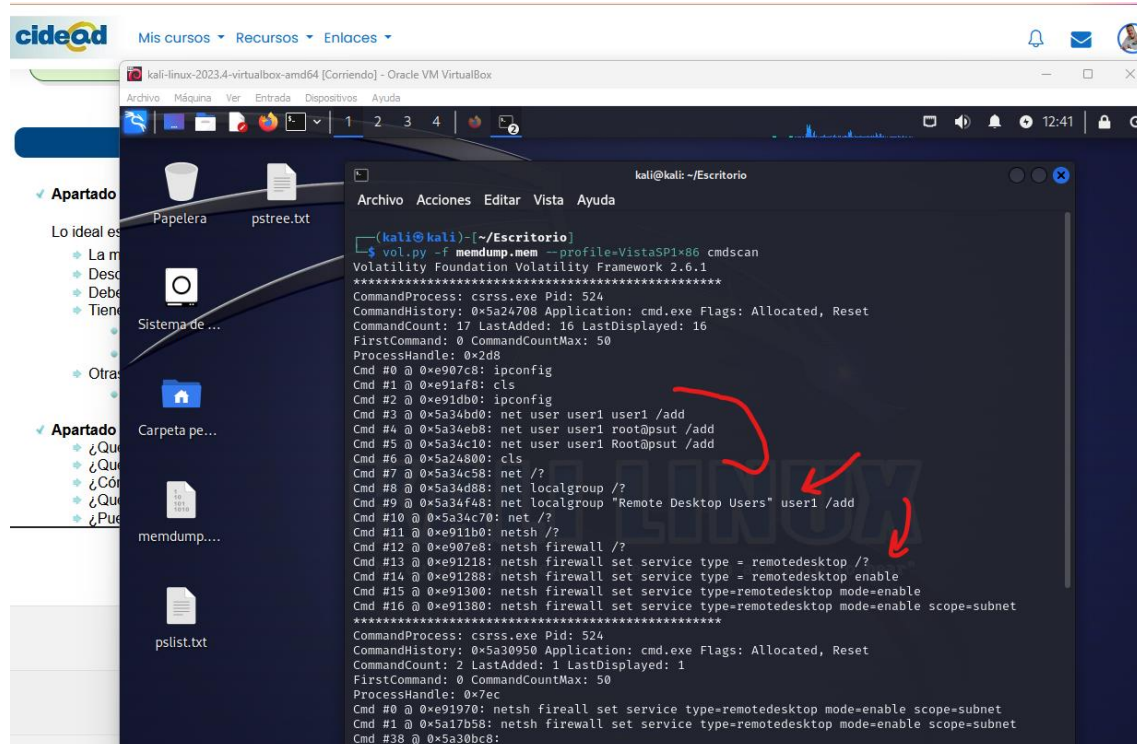
```
vol.py -f memdump.mem --profile=VistaSP1x86 cmdscan
```



```
kali@kali: ~/Escritorio
Archivo Acciones Editar Vista Ayuda

(kali@kali)~[~/Escritorio]
$ vol.py -f memdump.mem --profile=VistaSP1x86 cmdscan
Volatility Foundation Volatility Framework 2.6.1
*****
CommandProcess: csrss.exe Pid: 524
CommandHistory: 0x5a24708 Application: cmd.exe Flags: Allocated, Reset
CommandCount: 17 LastAdded: 16 LastDisplayed: 16
FirstCommand: 0 CommandCountMax: 50
ProcessHandle: 0x2d8
Cmd #0 @ 0xe907c8: ipconfig
Cmd #1 @ 0xe91af8: cls
Cmd #2 @ 0xe91db0: ipconfig
Cmd #3 @ 0x5a34bd0: net user user1 user1 /add
Cmd #4 @ 0x5a34eb8: net user user1 root@psut /add
Cmd #5 @ 0x5a34c10: net user user1 Root@psut /add
Cmd #6 @ 0x5a24800: cls
Cmd #7 @ 0x5a34c58: net /?
Cmd #8 @ 0x5a34d88: net localgroup /?
Cmd #9 @ 0x5a34f48: net localgroup "Remote Desktop Users" user1 /add
Cmd #10 @ 0x5a34c70: net /?
Cmd #11 @ 0xe911b0: netsh /?
Cmd #12 @ 0xe907e8: netsh firewall /?
Cmd #13 @ 0xe91218: netsh firewall set service type = remotedesktop /?
Cmd #14 @ 0xe91288: netsh firewall set service type = remotedesktop enable
Cmd #15 @ 0xe91300: netsh firewall set service type=remotedesktop mode=enable
Cmd #16 @ 0xe91380: netsh firewall set service type=remotedesktop mode=enable scope=subnet
*****
CommandProcess: csrss.exe Pid: 524
CommandHistory: 0x5a30950 Application: cmd.exe Flags: Allocated, Reset
CommandCount: 2 LastAdded: 1 LastDisplayed: 1
FirstCommand: 0 CommandCountMax: 50
ProcessHandle: 0x7ec
Cmd #0 @ 0xe91970: netsh firewall set service type=remotedesktop mode=enable scope=subnet
Cmd #1 @ 0x5a17b58: netsh firewall set service type=remotedesktop mode=enable scope=subnet
Cmd #38 @ 0x5a30bc8:
```

Encontramos que se ha accedido a la máquina, creado un usuario al que le han dado permisos y añadido para acceso remoto.

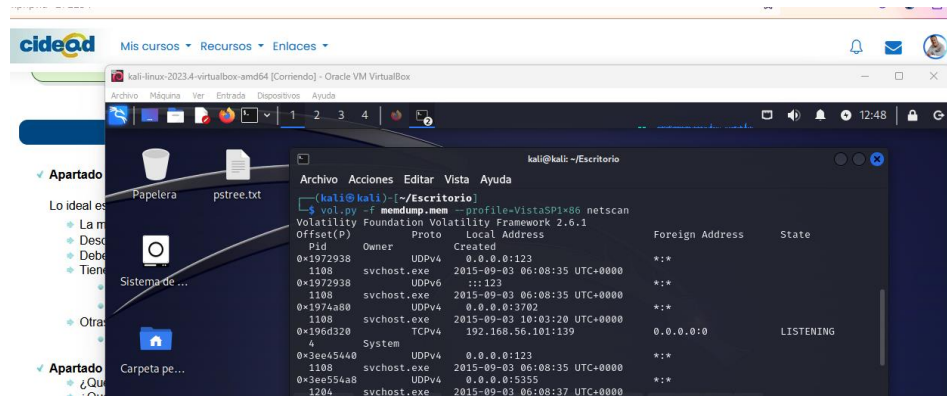


```
kali@kali: ~/Escritorio
Archivo Acciones Editar Vista Ayuda

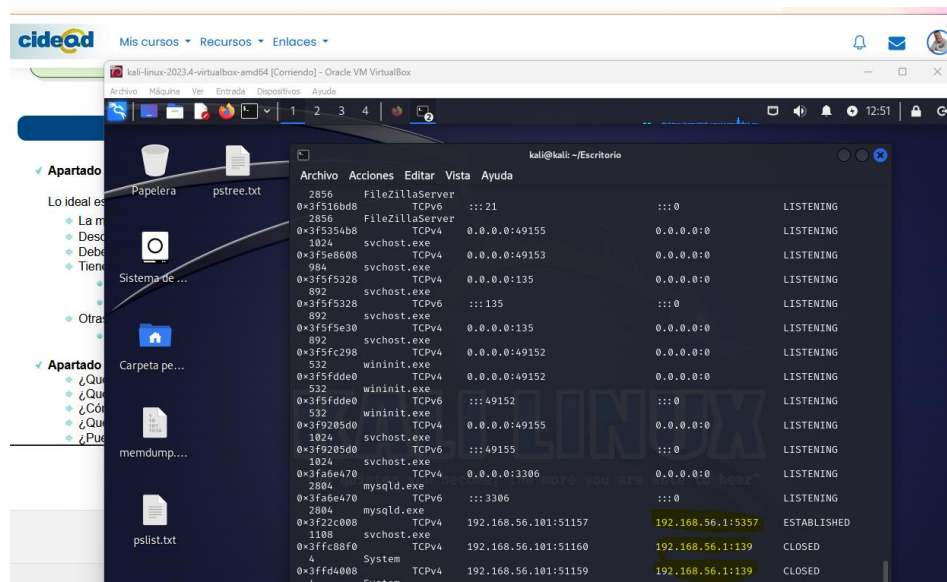
(kali@kali)~[~/Escritorio]
$ vol.py -f memdump.mem --profile=VistaSP1x86 cmdscan
Volatility Foundation Volatility Framework 2.6.1
*****
CommandProcess: csrss.exe Pid: 524
CommandHistory: 0x5a24708 Application: cmd.exe Flags: Allocated, Reset
CommandCount: 17 LastAdded: 16 LastDisplayed: 16
FirstCommand: 0 CommandCountMax: 50
ProcessHandle: 0x2d8
Cmd #0 @ 0xe907c8: ipconfig
Cmd #1 @ 0xe91af8: cls
Cmd #2 @ 0xe91db0: ipconfig
Cmd #3 @ 0x5a34bd0: net user user1 user1 /add
Cmd #4 @ 0x5a34eb8: net user user1 root@psut /add
Cmd #5 @ 0x5a34c10: net user user1 Root@psut /add
Cmd #6 @ 0x5a24800: cls
Cmd #7 @ 0x5a34c58: net /?
Cmd #8 @ 0x5a34d88: net localgroup /?
Cmd #9 @ 0x5a34f48: net localgroup "Remote Desktop Users" user1 /add
Cmd #10 @ 0x5a34c70: net /?
Cmd #11 @ 0xe911b0: netsh /?
Cmd #12 @ 0xe907e8: netsh firewall /?
Cmd #13 @ 0xe91218: netsh firewall set service type = remotedesktop /?
Cmd #14 @ 0xe91288: netsh firewall set service type = remotedesktop enable
Cmd #15 @ 0xe91300: netsh firewall set service type=remotedesktop mode=enable
Cmd #16 @ 0xe91380: netsh firewall set service type=remotedesktop mode=enable scope=subnet
*****
CommandProcess: csrss.exe Pid: 524
CommandHistory: 0x5a30950 Application: cmd.exe Flags: Allocated, Reset
CommandCount: 2 LastAdded: 1 LastDisplayed: 1
FirstCommand: 0 CommandCountMax: 50
ProcessHandle: 0x7ec
Cmd #0 @ 0xe91970: netsh firewall set service type=remotedesktop mode=enable scope=subnet
Cmd #1 @ 0x5a17b58: netsh firewall set service type=remotedesktop mode=enable scope=subnet
Cmd #38 @ 0x5a30bc8:
```

Vistos esos datos, buscamos en las conexiones para ver si hay alguna externa

vol.py -f memdump.mem -profile=VistaSP1x86 netscan

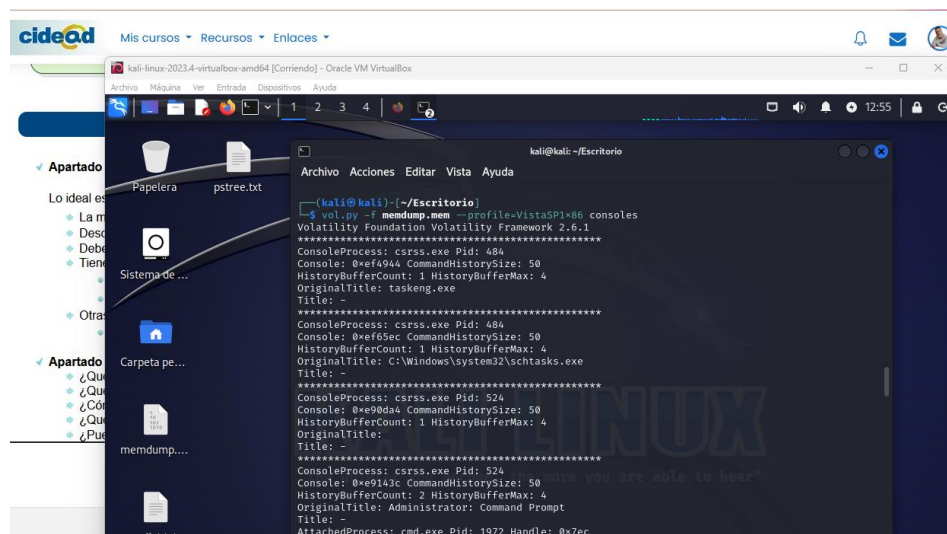


Aparecen entre otros datos, una dirección IP distinta: **192.168.56.1** que podría ser el router.



El siguiente comando con el que intentaremos encontrar información sobre las consolas utilizadas es:

vol.py -f memdump.mem -profile=VistaSP1x86 consoles

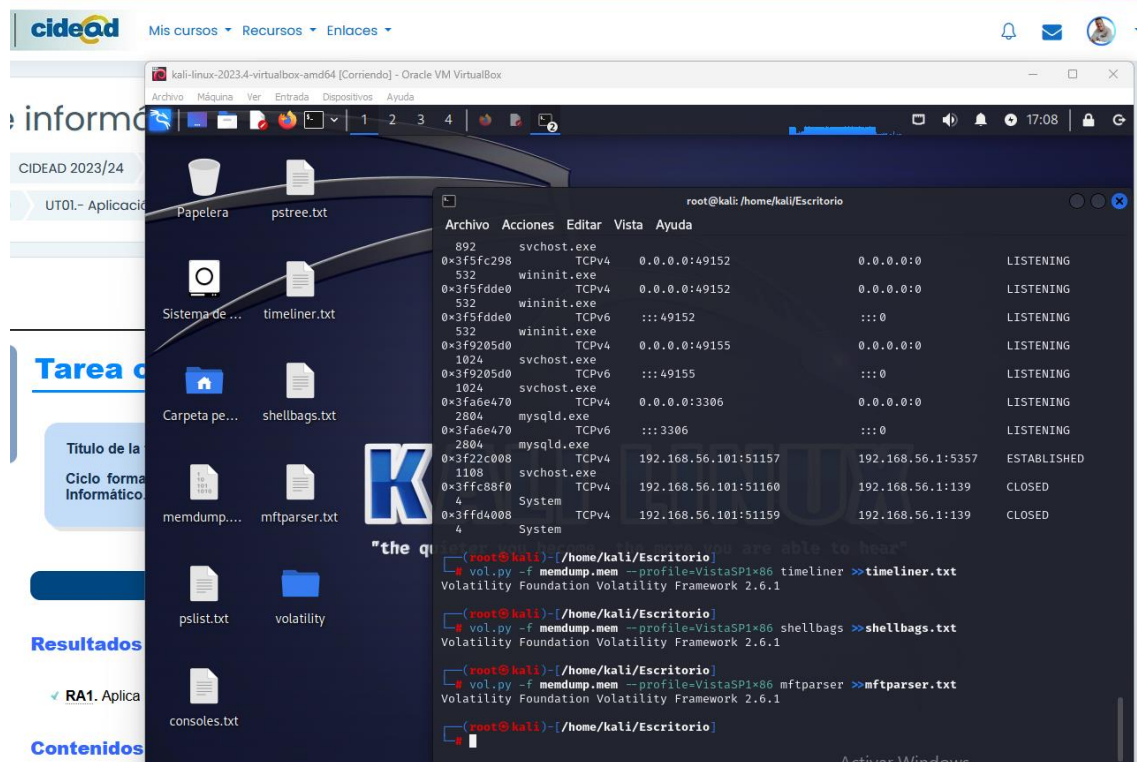


Lo volcamos también a un archivo de texto y antes de indagar en él, recogemos más datos para cotejar con:

```
vol.py -f memdump.mem --profile=VistaSP1x86 timeliner >>timeliner.txt
```

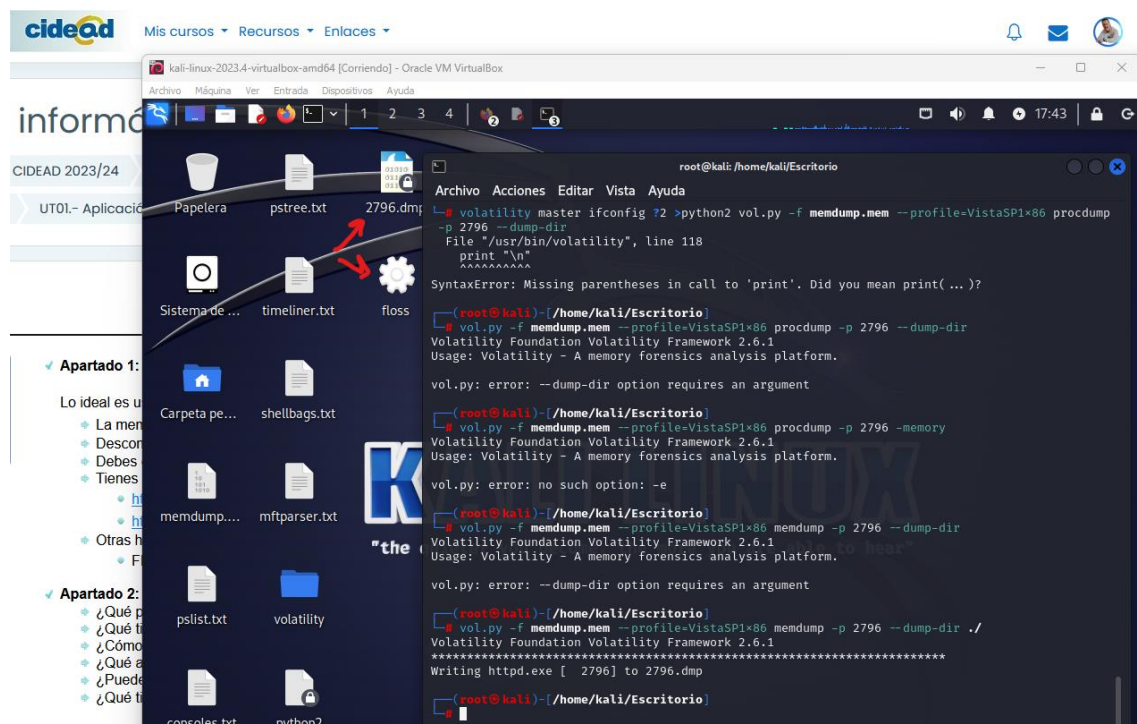
```
vol.py -f memdump.mem --profile=VistaSP1x86 shellbags >>shellbags.txt
```

```
vol.py -f memdump.mem --profile=VistaSP1x86 mftparser >>mftparser.txt
```

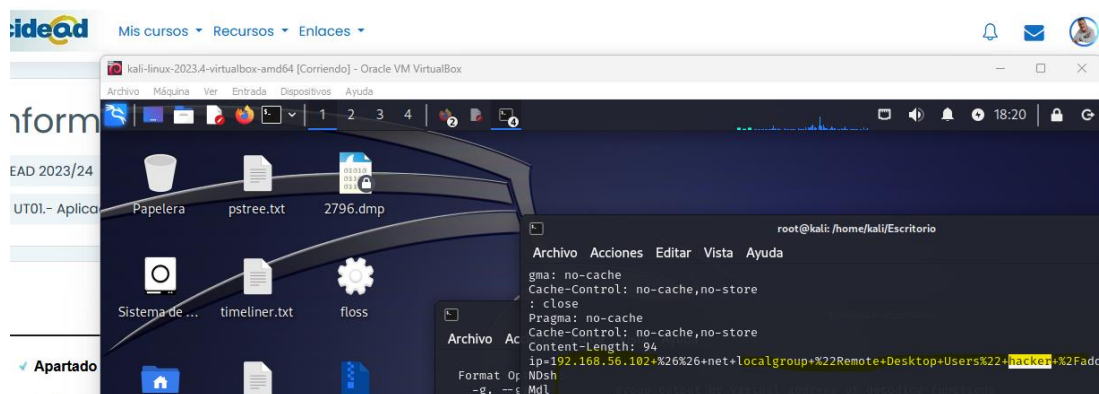


Hago un volcado de los procesos de httpd con PID 2796

```
vol.py -f memdump.mem --profile=VistaSP1x86 memdump -p 2796 --dump-dir ./
```

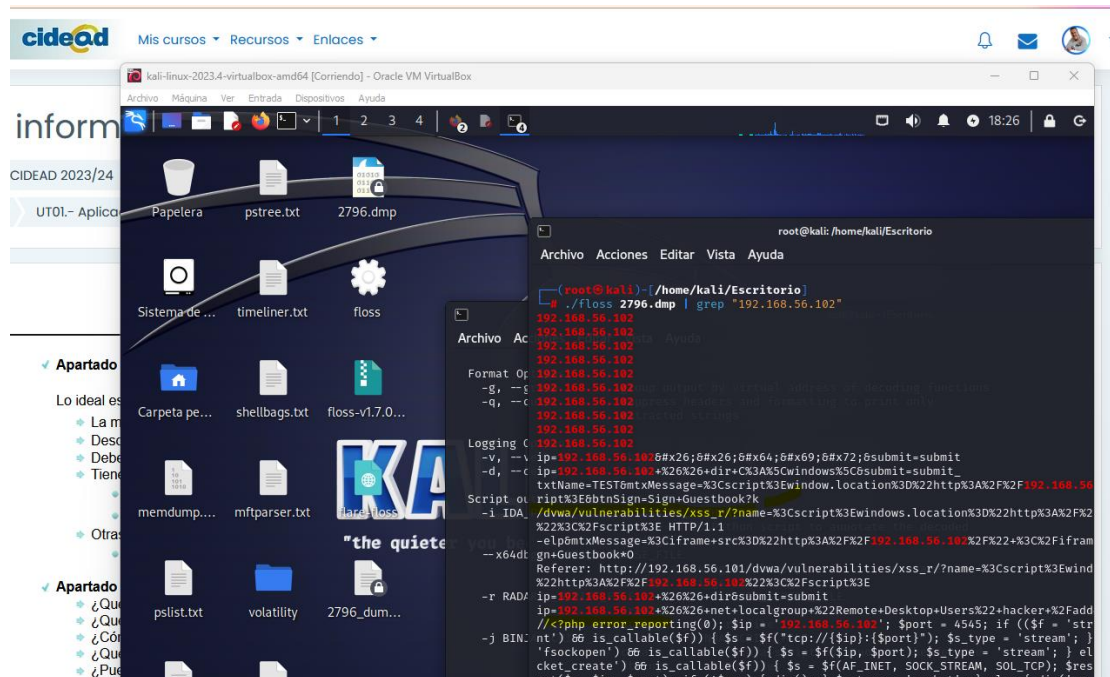



```
./floss 2796.dmp
```

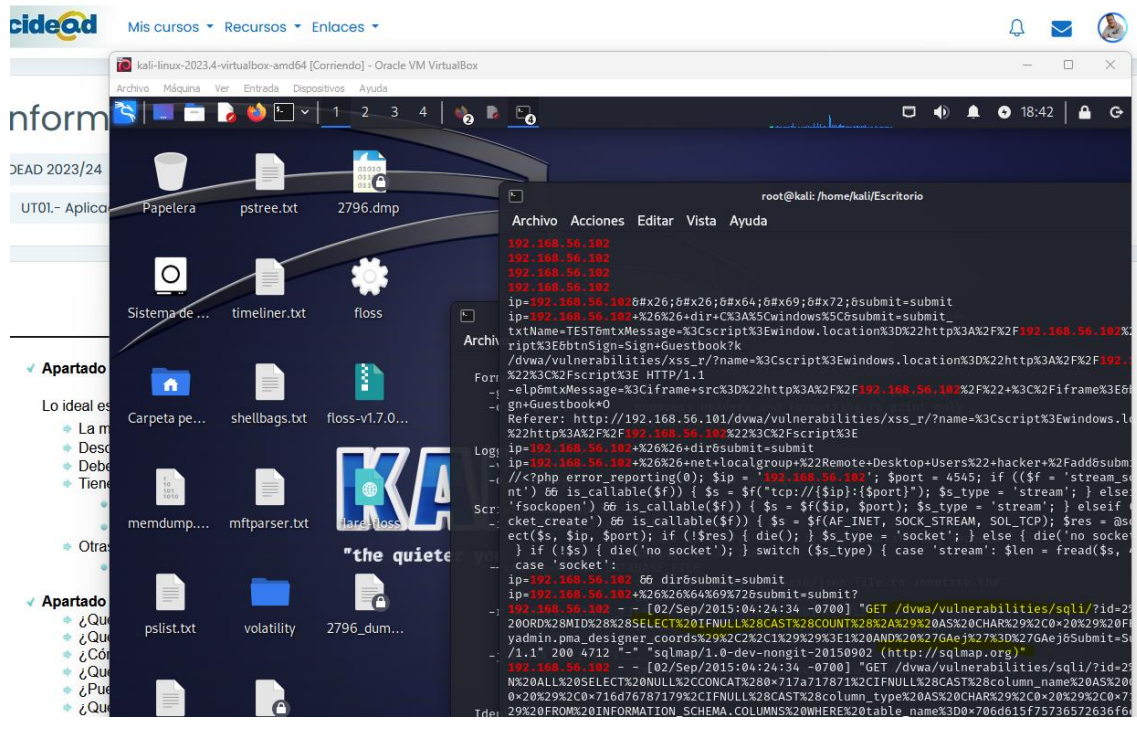


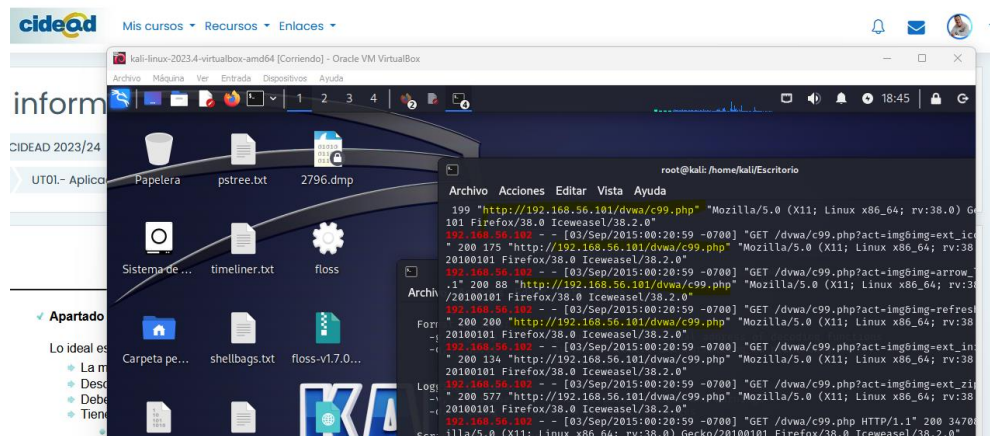
Encontradas sesiones con nueva IP, usuario **Hacker** y credenciales hacia **dvwa** (aplicación vulnerable que suele utilizar esta IP como escucha para los ataques). Cotejo esa IP con el archivo anterior.

`./floss 2796.dmp | grep "192.168.56.102"`



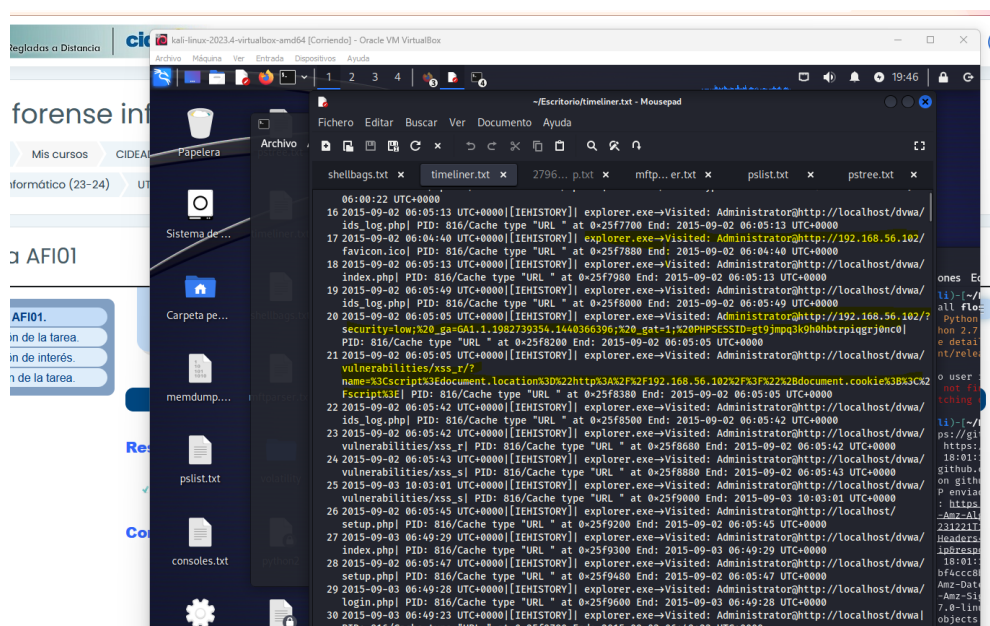
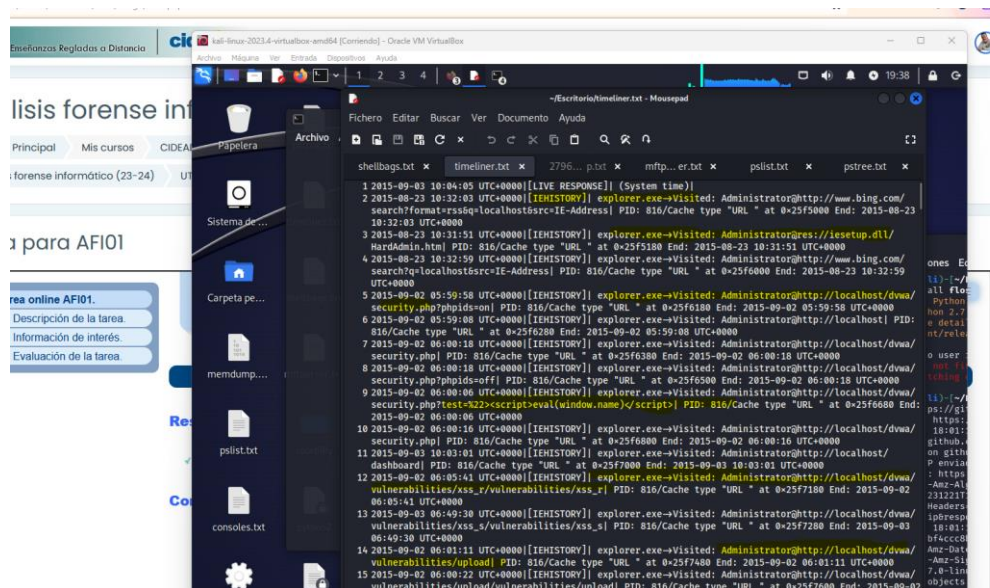
Encontramos múltiples coincidencias, junto a rastro de acciones, muchas habituales en **dvwa**.

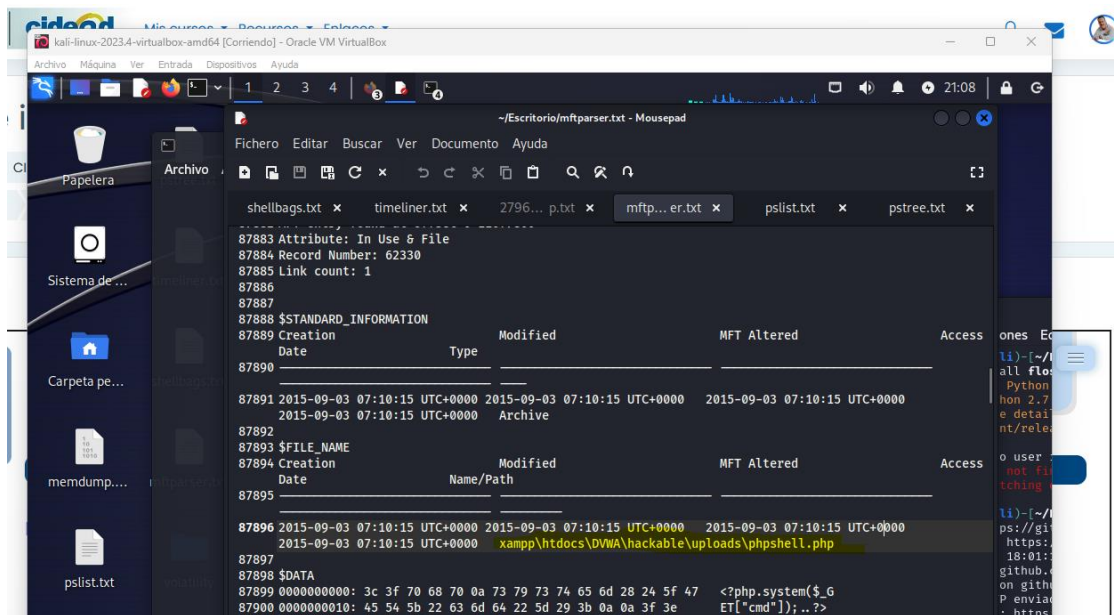




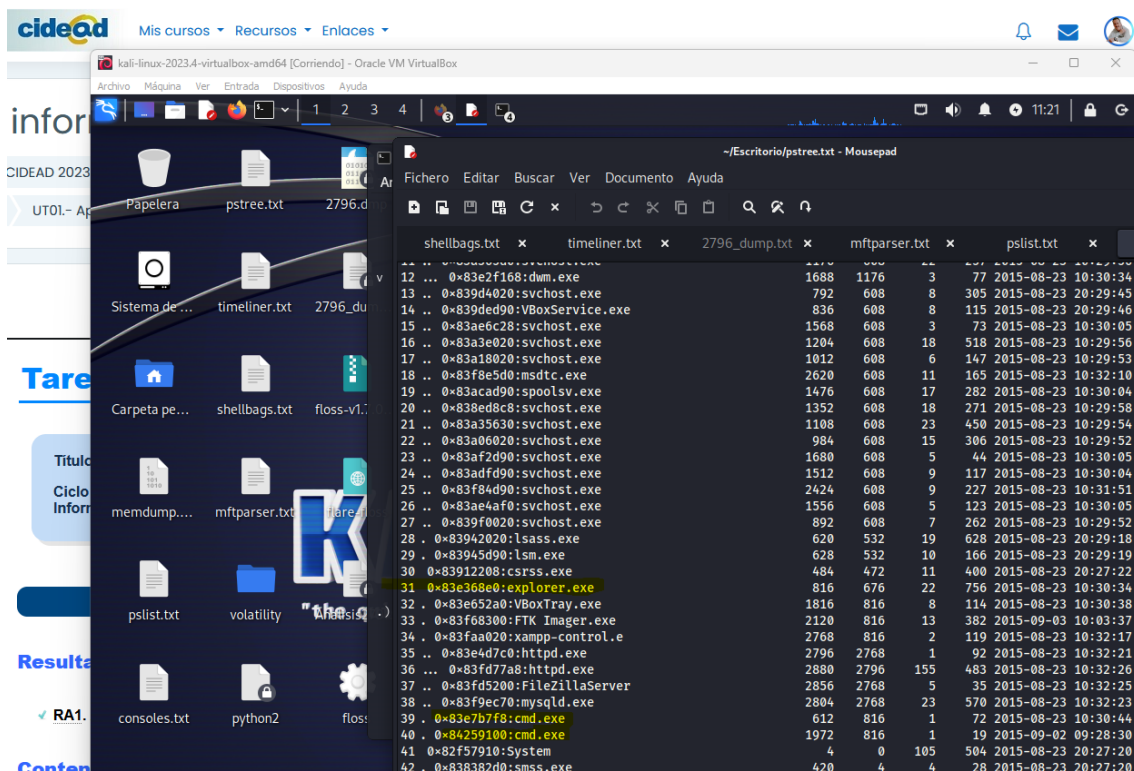
Se observa en las capturas anteriores una repetición de peticiones desde esta IP a nuestro servidor, incluyendo el archivo **c99.php** (webshe11).

Si revisamos los archivos, podemos observar como ha accedido desde **IE** al sitio de **dwa** y el resto de las acciones, incluida la subida de una **phpshe11** donde estaría el **c99.php**.



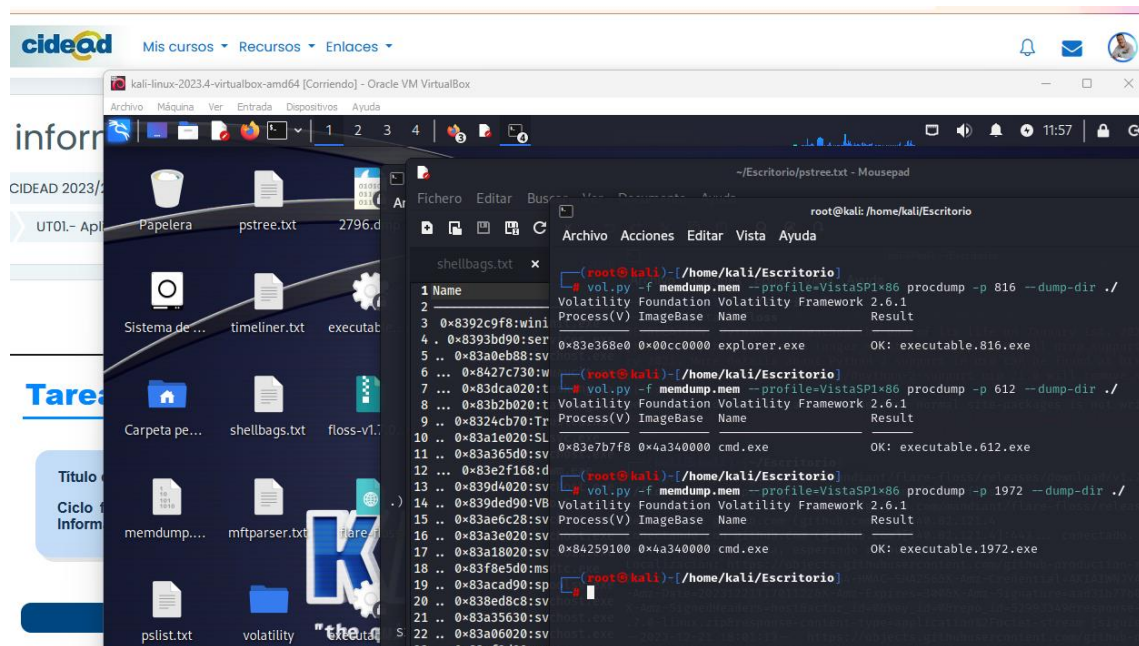


Vistas las coincidencias en estos procesos y archivos, además del **PiD 2796**, creo que debemos explorar el utilizado por **cmd.exe** o por **explorer.exe**

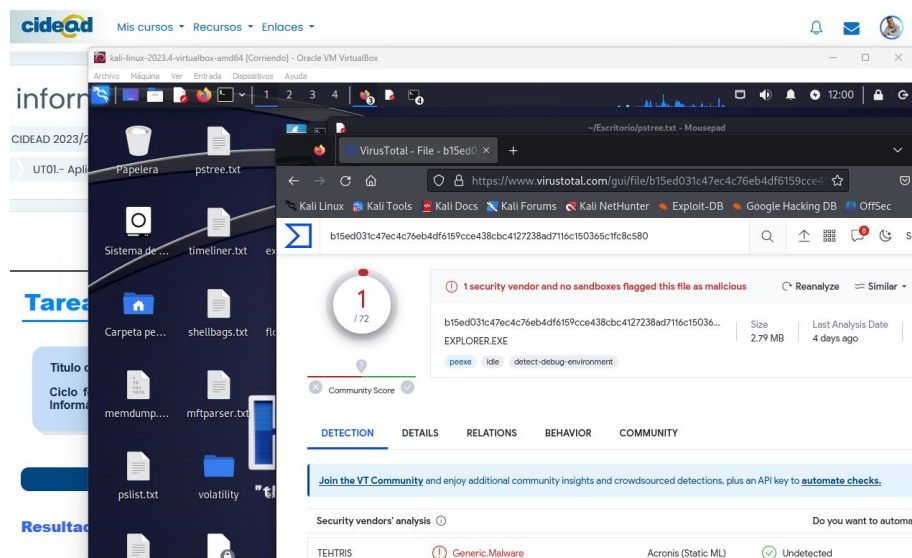


Empezamos por **explorer.exe** y finalmente con **cmd.exe**

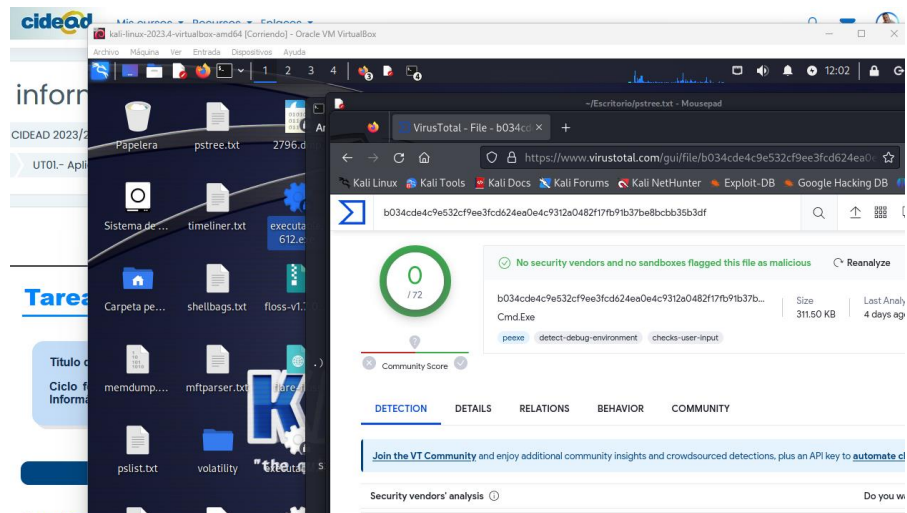
```
vol.py -f memdump.mem -profile=VistaSP1x86 procdump -p 816 -dump-dir ./
vol.py -f memdump.mem -profile=VistaSP1x86 procdump -p 612 -dump-dir ./
vol.py -f memdump.mem -profile=VistaSP1x86 procdump -p 1972 -dump-dir ./
```

Explorer.exe nos arroja resultados en Virustotal sobre un malware genérico.

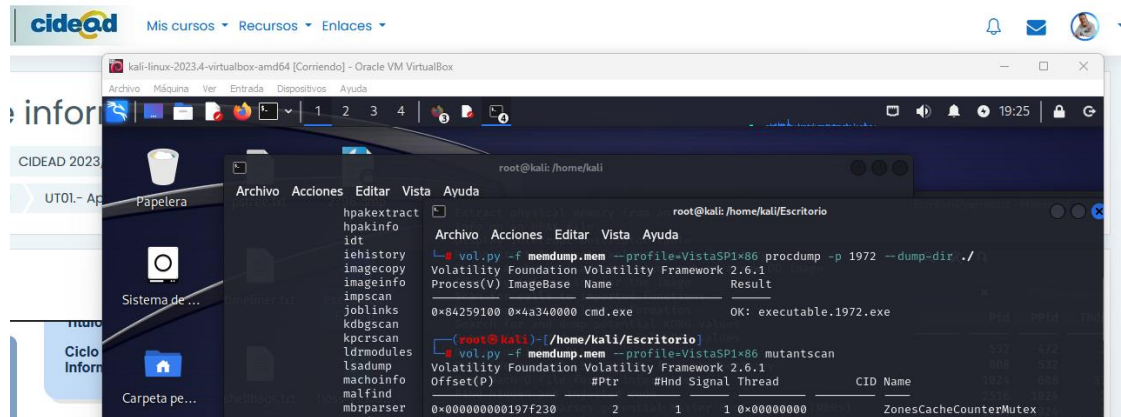


Los otros dos ejecutables, salen limpios.

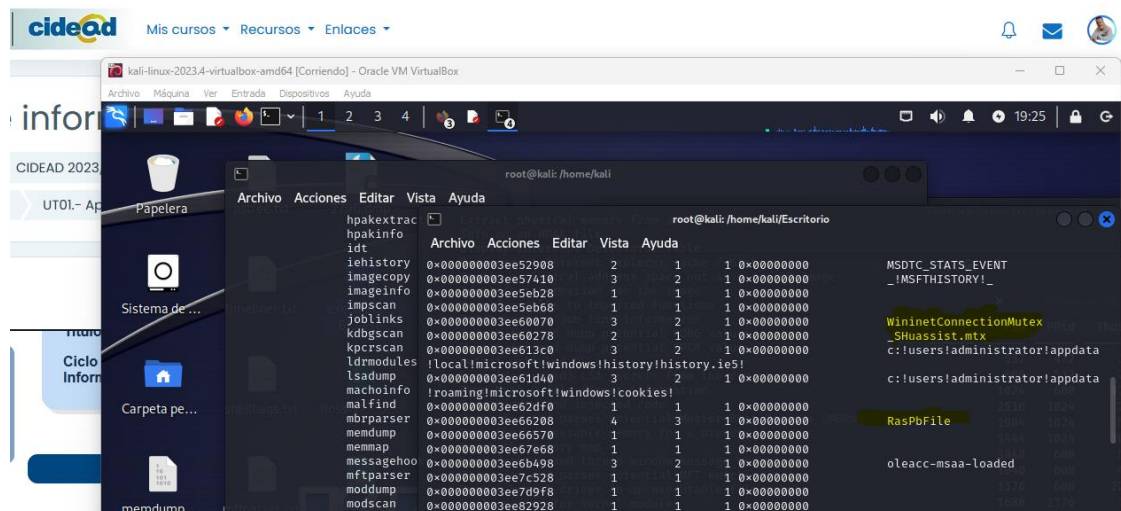


Buscamos por último algún rastro más de las acciones mediante **mutantscan**, buscando objetos mutex u objetos de exclusión mutua, que muchas veces se utilizan con el mismo propósito que lo hacen los procesos legítimos, que es garantizar que se ejecuta el código (en este caso sería un virus) sin que otro escriba en su espacio de memoria.

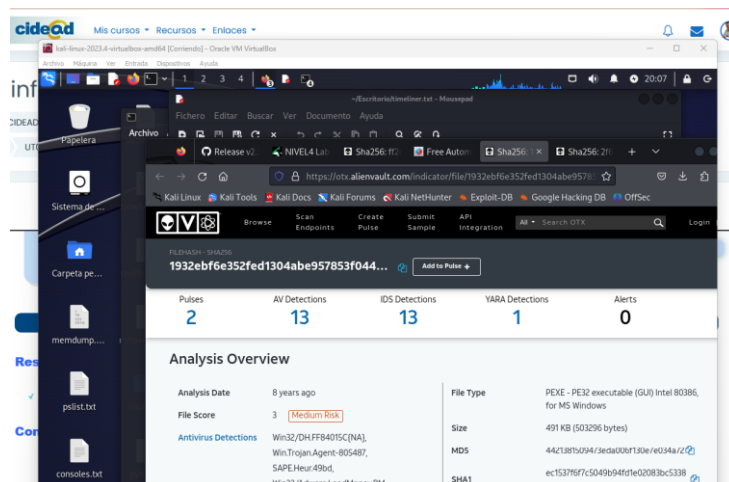
vol.py -f memdump.mem --profile=VistaSP1x86 mutantscan



Aparecen referencias utilizadas habitualmente para infectar sistemas como **WininetConnectionMutex_SHUassist.mtx** o **RaspFile** (este último involucrado con el anterior).



Si buscamos algo de información sobre ello, siempre aparecen relacionados con Troyanos.



Webgrafía.

<https://www.volatilityfoundation.org/releases>

<https://seanthegeek.net/1172/how-to-install-volatility-2-and-volatility-3-on-debian-ubuntu-or-kali-linux/>

https://www.youtube.com/watch?v=Yar_cD-XwqM

<https://classroom.anir0y.in/post/vol2-installation/>

<https://byte-mind.net/analisis-forense-con-volatility/>

<https://github.com/mandiant/flare-floss>

<https://lightrun.com/answers/mandiant-flare-floss-how-to-install-use-floss-flare-obfuscated-string-solver-in-ubuntu>

<https://github.com/mandiant/flare-floss/tree/master/doc>